

The perceptron, and the wall it hit

In 1958 a single artificial neuron was front-page news. In 1969 a short proof about four points helped freeze the field for a decade. Today you watch that proof happen live, on the neuron you built on Tuesday — and then see the one change that gets through the wall.

Unit 2 · From scratch

80 minutes

Demonstration, not a lab

Deck slide 26 (v2)

1958 The XOR wall 1969 The fix Before Tuesday

PART 1 · 15 MINUTES

Rosenblatt, 1958: a machine that learns

Session 3 told this story in one line. Today it gets its own session.

1

Frank Rosenblatt's perceptron was a single neuron built in hardware, with adjustable weights, that learned to classify simple patterns from examples. The press reported it as the embryo of a machine that would walk, talk and be conscious of its existence. The mathematics was modest and correct; the promises were not.

THE PERCEPTRON LEARNING RULE

$$w \leftarrow w + (y - \hat{y}) \cdot x \quad b \leftarrow b + (y - \hat{y})$$

Predict 1 if the score is positive, else 0. When the prediction is wrong, nudge the weights towards the right answer; when it is right, do nothing.

Compare it with Tuesday's update, $w \leftarrow w - \eta(p - y) \cdot x$. It is the same shape with the sign folded in and a hard step in place of the sigmoid. Rosenblatt proved something genuinely important: **if a separating line exists, this rule is guaranteed to find one** in a finite number of mistakes.

the perceptron learns AND

PYTHON

```
X = np.array([[0,0],[0,1],[1,0],[1,1]]); y_and = np.array([0,0,0,1])
w, b = np.zeros(2), 0.0
for epoch in range(10):
    for xi, yi in zip(X, y_and):
        pred = int(w @ xi + b > 0)
        w, b = w + (yi - pred) * xi, b + (yi - pred)
print([int(w @ xi + b > 0) for xi in X])
```

output

OUTPUT

[0, 0, 0, 1]

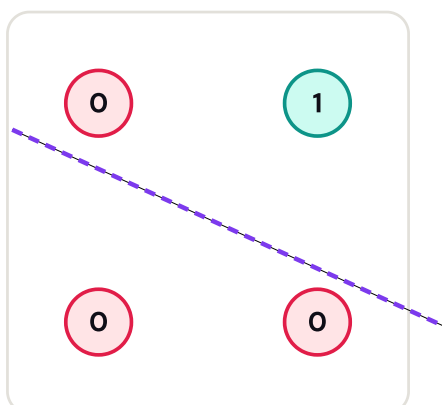
PART 2 · 20 MINUTES · LIVE

Four points, and a line that cannot be drawn

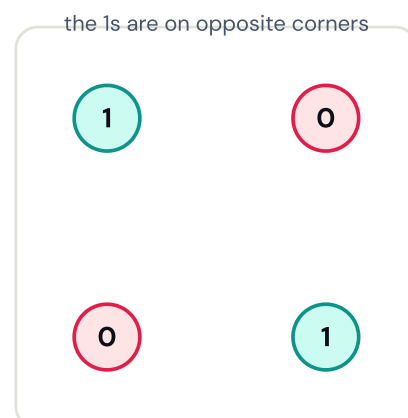
Run at the front from CSC4315_S12_Demo_The_XOR_Wall.

2

YOUR HOMEWORK FROM TUESDAY, DRAWN OUT



AND — one line works



XOR — try it

XOR says yes when exactly one input is on. Put the four cases on the corners of a square and the two yes points sit on one diagonal, the two *no* points on the other. Any straight line that puts both 1s on one side puts at least one 0 there too. You tried; everybody does.

The demonstration

We take Tuesday's neuron — sigmoid, cross-entropy, gradient descent — start it from small random weights so the failure cannot be blamed on starting at zero, and train it for five thousand epochs on the four XOR points.

what the room sees

TEXT

```
probabilities: [0.5 0.5 0.5 0.5]
loss: 0.6931    log 2 = 0.6931
```

THIS IS NOT A BUG

The neuron is doing its honest best. The line that minimises cross-entropy on these four points is the one that refuses to decide: $p = 0.5$ everywhere, $\text{loss} = \log 2$. More epochs, a different learning rate, a cleverer initialisation — none of it helps. **The limit is in the model, not in the training.**

PART 3 · 10 MINUTES

Minsky and Papert, 1969

How a correct theorem became an over-read verdict.

3

Marvin Minsky and Seymour Papert's book *Perceptrons* proved, carefully, what the demonstration just showed: a single-layer perceptron cannot represent XOR, nor a family of related problems that depend on how inputs combine. They knew multi-layer networks might escape the limit; what nobody had was a practical way to *train* one.

The theorem was read far more broadly than it was written, funding dried up, and neural networks spent the next decade and more in the first AI winter from session 3. The way out — training the hidden layers — was backpropagation, popularised in 1986. You will derive it in session 14.



The historical slide from session 3 pays off here

Deck slide 26 (v2) put the perceptron and the first winter on one timeline. You now know exactly which four points were on that slide, and why they mattered.

PART 4 · 20 MINUTES

Through the wall: one hidden layer

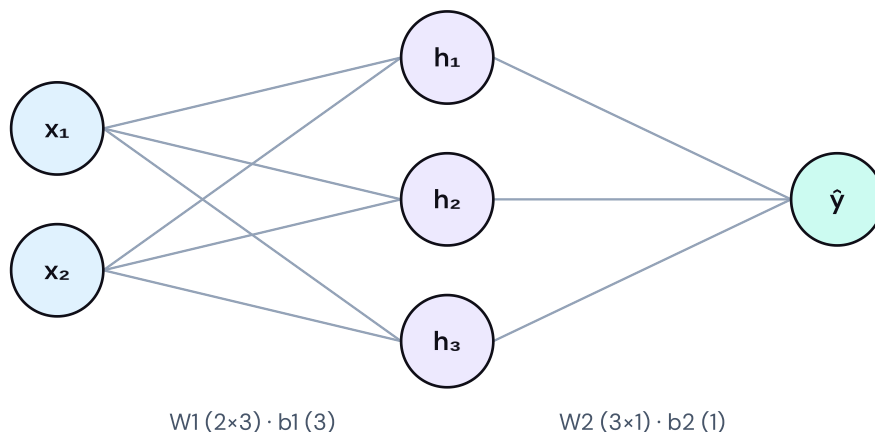
Motivated by the failure, not asserted.

4

If one line is not enough, use several and combine their answers. Put three sigmoid neurons between the inputs and the output. Each one draws its own line across the square; the output neuron then combines three yes-or-no answers — and **a combination of lines is not a line**.

THE 2-3-1 NETWORK: THE ARCHITECTURE THAT STAYS WITH US TO DECEMBER

two inputs → three hidden neurons → one output



hand-set weights, shown in the demonstration

PYTHON

```
W1 = np.array([[4., 4., -4.], [4., -4., 4.]])  b1 = np.array([-2., 2., 2.])
W2 = np.array([[6.], [-6.], [-6.]])          b2 = np.array([3.])
A1 = sigmoid(X @ W1 + b1)
out = sigmoid(A1 @ W2 + b2)
print(np.round(out.ravel(), 3))
```

output

OUTPUT

[0.001 0.83 0.83 0.17]

Zero, one, one, zero — XOR, near enough. These weights were chosen by hand, which is cheating in an honest way: the point today is only that *such weights exist*. Two questions remain, and they are the next two sessions. **Tuesday:** how exactly does an input travel through this network? **Friday:** how would we *find* those thirteen numbers from data?

WHY LAB 5'S COLLAPSE DOES NOT HAPPEN HERE

Lab 5 proved that two linear layers stacked are just one linear layer. What breaks that collapse is the sigmoid between them. **Without a non-linear activation, no number of hidden neurons could solve XOR.** That is the real reason activation functions exist.

Three questions for the room

Discuss with your neighbour first.

Q1. Why does the single neuron settle on 0.5 rather than getting three of four right? Three-of-four with one confident mistake costs more cross-entropy than four coin tosses: one confidently wrong point is very expensive. The loss prefers honest uncertainty.

Q2. Count the parameters in the 2–3–1 network. W1 has 6, W2 has 3: nine weights. b1 has 3, b2 has 1: four biases. Thirteen numbers — deck slide 21.

Q3. Is the perceptron rule wrong? No. It is correct for what it is — a guaranteed method for problems where a line exists. The wall was a limit of one layer, not of learning.

Before Tuesday

There is no lab today. Instead:

1. **Rerun the demonstration notebook** yourself — `CSC4315_S12_Demo_The_XOR_Wall`.
2. **Without notes:** the perceptron rule, why XOR defeats one line, and the parameter count of the 2–3–1 network.
3. **Bring a calculator** on Tuesday. You will compute every number in that network by hand.



Session 13 — Tuesday 20 October, 16:00

Forward pass through a 2–3–1 network: nine weights, four biases, one example — on paper first, then in three lines of NumPy, checked digit by digit.